DT-as-a-Service on the RADICAL stack

What DT-as-a-Service means for NSTX-U · scoping the first use case · August 2026

RADICAL, Rutgers University

Digital twins as long-running services on DOE HPC.
v1 is implemented and measured.

What is a digital twin (here)

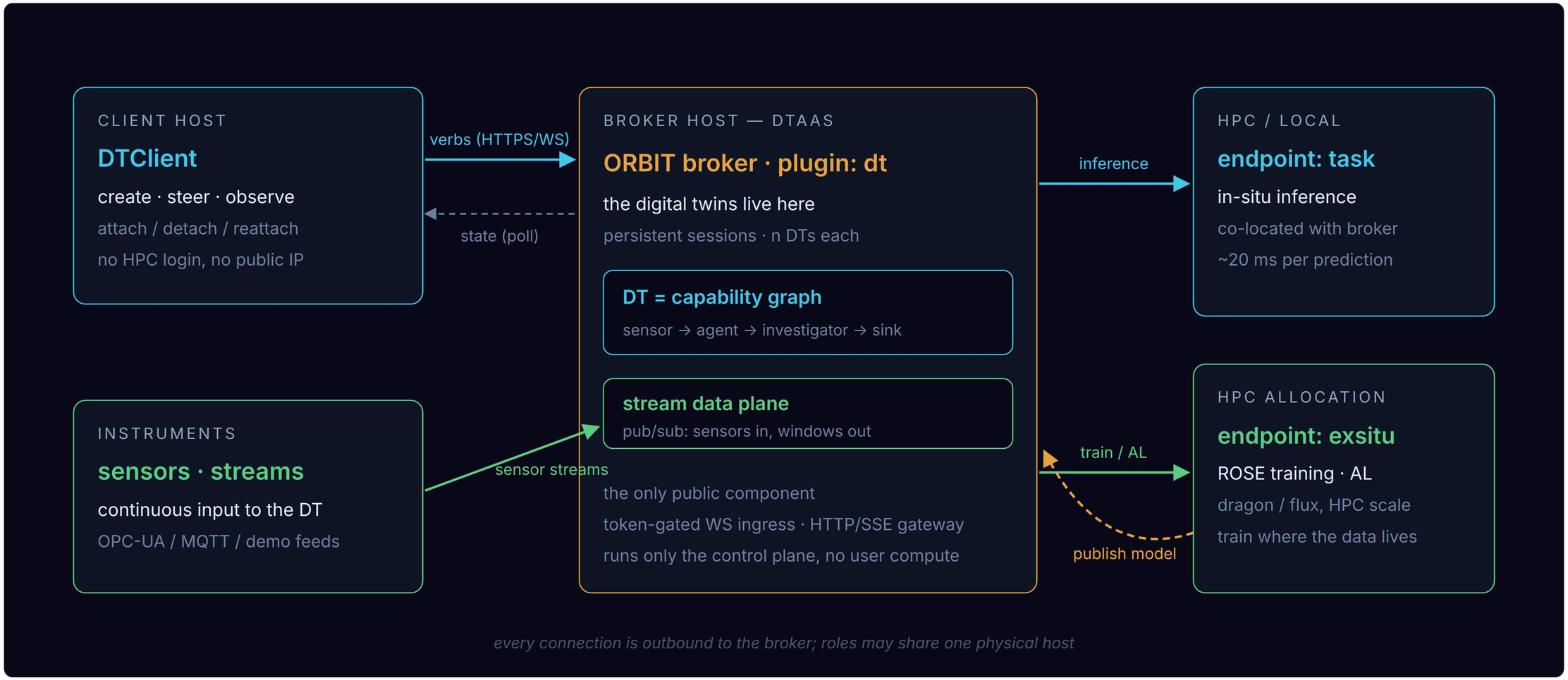
- A digital twin is a computational counterpart of a physical system: its models **keep learning** against reality while **serving inference**.
- The DT is a **graph of capabilities**: typed input → output contracts (e.g. diagnostics → heat flux), each realized by one or more learned models.
- Two concurrent loops: **learning** runs until its objective is met and restarts on new data; **inference** runs indefinitely and hot-swaps models as they improve.
- Event sources drive the DT: sensors, shot data, simulation output, users, timers.

What "as a service" adds

- The DT control plane runs as a plugin on a **persistent ORBIT broker**; the DT runs for months, independent of any login session.
- Scientists **attach from a laptop**, steer, and detach; the DT keeps running. Reattach later, from anywhere.
- One broker hosts **multiple sessions** (name spaces); each session holds **multiple DTs**.
- Compute is acquired **on demand**: inference and training tasks run in HPC allocations through the AmSC/RADICAL stack.
- Instrument streams feed the DT continuously; all connections are **outbound**, the facility opens no inbound ports.

```
L5 DT framework → L4 ROSE → L3 AsyncFlow → L2.5 ORBIT → L2 Rhapsody → L1 resources
```

Architecture — roles



Four roles: the client steers, instruments feed, the broker hosts the digital twins, endpoints compute. The broker is the only publicly reachable component.

Status

- **DTaaS v1 is implemented** on the RADICAL stack and under review: the ORBIT dt plugin, a streaming learner, and two stream backends.
- **126 tests**, including live-broker integration; demos run end to end and **calibration converges**.
- Latencies are **measured**: ~20 ms per served prediction, 1–2 ms per stream hop.
- A live **viz dashboard** shows convergence and activity, with live and replay modes.

Architecture, lifecycle, benchmarks, and the requirements map are in the backup slides.

What this offers NSTX-U

- A surrogate on NSTX-U geometry **has been demonstrated on this stack**: the heat recipe learns the Eich optical heat flux with active learning on Perlmutter.
- No standing service is required: the compute side is **spawned on demand** into a normal allocation and torn down after the run.
- Diagnostics become **event sources**: any stream feeds the DT, live or from shot archives.
- Models **learn between shots** and **serve between and during shots**; they improve continuously as data arrives.
- Compute runs on DOE HPC; PPPL deploys **no services** and opens **no inbound ports**.
- Every run is tracked (MLflow); results and models stay inspectable.

First use case — a World Model for the NSTX-U DT

- Proposal: scope this together, starting from five questions.
- **Meaning**: is the world model one learned model of the machine state which serves many capabilities, or a graph of per-capability surrogates around a shared state estimate?
- **Signals**: which diagnostics come first, and at what rate?
- **Data**: shot archives, live shots, simulation (XGC, M3DC1), or a mix?
- **Placement and latency**: where does learning run; must serving keep up between shots or within a shot?
- **Success**: what does a first demo have to show?

The framework is use-case independent; these choices configure it and do not change it.

Discussion · next steps

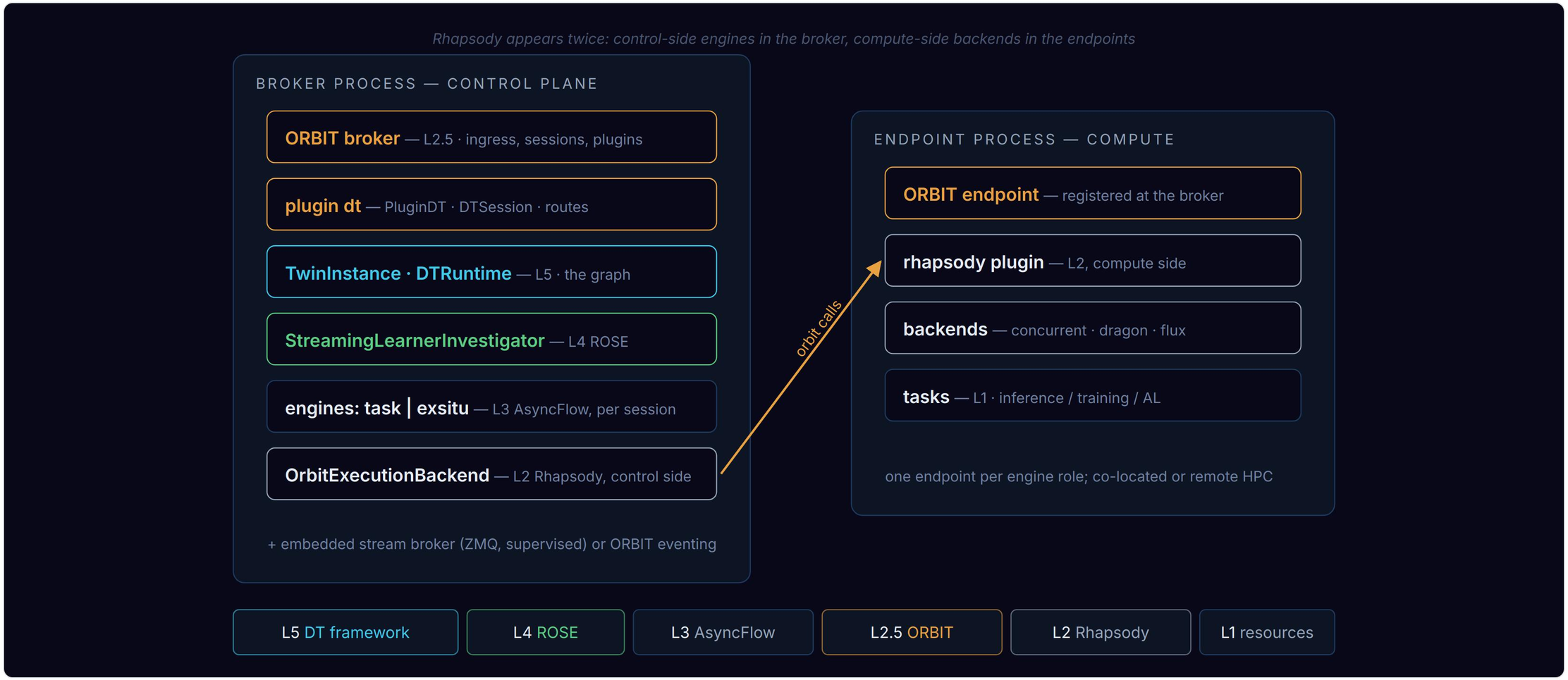
- Agree on the world-model scope and the first capability.
- Identify data access: which archives and streams, and who provides them.
- Pick a first demo target and date; the service stack is ready to host it.

References: dtaas-v1-plan.md (implementation) · amsc/architecture/dt-framework.md (architecture) · use-cases/heat (NSTX-U heat-flux recipe)

Backup

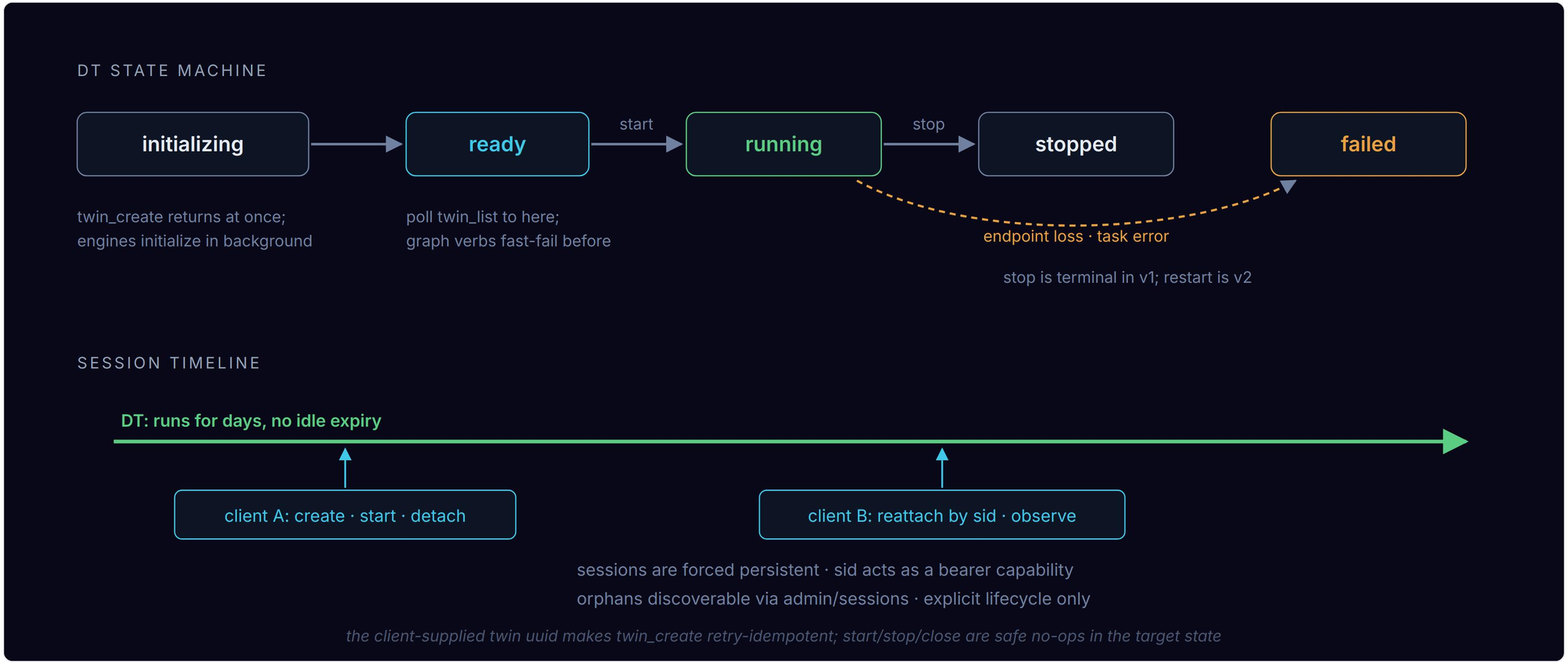
Architecture detail · lifecycle · data plane · implementation · benchmarks · requirements

Architecture — software



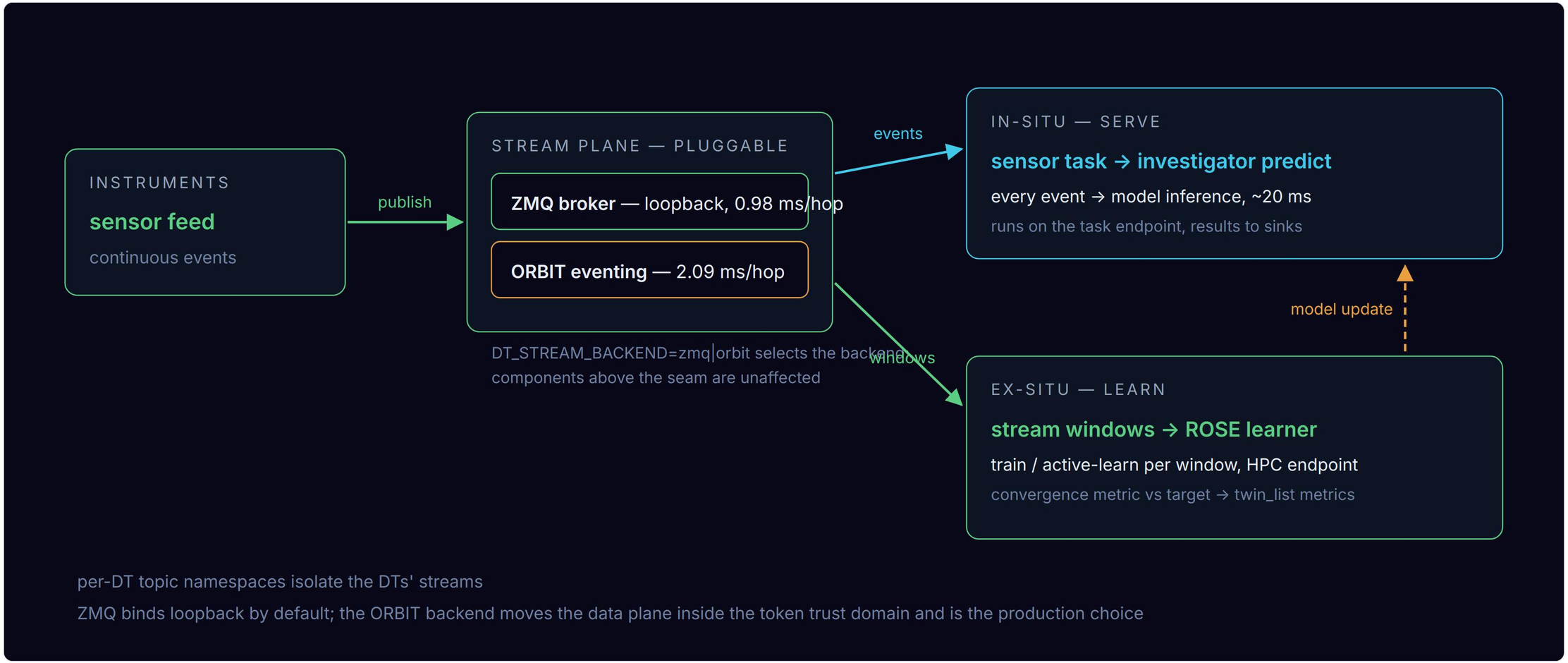
New v1 code is the dt plugin and the hardened L5 runtime; the layers below are reused unchanged.

Lifecycle — DT states and sessions



twin_create is the only asynchronous verb; all other verbs are short and synchronous. Clients attach and detach; the digital twin continues to run.

Data plane — streams and learning



One stream feeds two loops: serving runs on every event, learning on every window; the model update closes the learning cycle.

Implementation status

MILESTONE	DELIVERS	STATE
M0 hardening	stop/teardown, per-DT topic namespaces, embeddable stream broker, packaging, pytest/tox	**PR ready**
M1 ORBIT plugin	**dt** plugin: persistent sessions, sid reattach, async twin_create + short sync verbs, admin listing	**PR ready**
M2 ROSE ex-situ	StreamingLearnerInvestigator, dual-engine DTs (task | exsitu), endpoint-loss visibility	**PR ready**
M3 data plane	ORBIT-pubsub backend: stream traffic inside the token domain	**PR ready**

- **126 tests**, including live-broker integration. Demos verified with both stream backends: service DT and streaming learner; **calibration converges**.
- Upstream: two ORBIT PRs **merged**; one AsyncFlow fix + one ROSE dependency in review.
- Live **viz dashboard**: convergence bars and activity lanes, in the ORBIT Explorer and standalone, with live and replay modes.

Measured performance

MEASUREMENT	RESULT	DECISION IT SETTLED
in-situ prediction	~20 ms p50 via endpoint vs ~11 ms in-process; endpoint wins under concurrency	**all** compute goes through Rhapsody, without an in-process pool; a co-located endpoint is the "local" deployment
stream hop	0.98 ms ZMQ · 2.09 ms ORBIT	the secure ORBIT data plane costs ~1 ms per hop and becomes the production default
sequential latency	261 ms → 20 ms after fixing the rhapsody notify batcher	fixed upstream as an endpoint configuration knob

Benchmarks live in the repo (perf/) and rerun against the demos.

Map to use-case requirements

REQUIREMENT (AMSC · XGFABRIC)	DTAAS V1 ANSWER
DT outlives any login session	**persistent sessions; DTs survive clients, reattach by sid**
drive from a laptop	**outbound-only ORBIT connectivity; no HPC login, no public IP**
train where the data lives	**exsitu engine → Rhapsody endpoint in the HPC allocation (dragon / flux)**
continuous sensor streams	**stream plane with per-DT namespaces; ZMQ or ORBIT backend**
serve inference continuously	**in-situ loop on the task endpoint, ~20 ms per prediction**
DOE-site security posture	**token-gated ingress; M3 puts stream traffic in the same trust domain**
many DTs, many users	n DTs per session, admin listing; **per-tenant auth is v2**
model selection · capability graphs · recovery	**v2**: cross-model selector, declarative graph spec, restart across broker loss